REORDER++: Enhanced Randomized Real-Time Scheduling Strategy Against Side-Channel Attacks

Jiankang Ren, *Member, IEEE*, Zheng Wang, Chi Lin, *Senior Member, IEEE*, Mohammad S. Obaidat, *Life Fellow, IEEE*, Hongrui Xie, Haihui Zhu, Chunxiao Liu, Kaiwen Wang, and Guozhen Tan

***Abstract*—Embedded real-time systems are widely adopted in safety-critical domains such as aircrafts, automobiles and space vehicles. Unfortunately, with the sharp rise in the use of common-off-the-shelf components in systems and the drive towards remote communication through untrusted networks, such as WiFi, radio or cellular, the security is increasingly becoming the key consideration in real-time system design. In particular, the real-time system is vulnerable to side-channel attacks from the external networks, which attempt to infer the timing of task execution by exploiting the system deterministic execution patterns. In this article, we present an enhanced online randomized scheduling strategy (named REORDER++), which breaks the deterministic task execution pattern of systems by random priority inversions at run-time to counteract the timing side-channel attacks in dynamic-priority real-time systems. In order to realize the feasible priority inversions under real-time constraint, we propose an online priority inversion test to increase the opportunity of tasks' priority inversions by judging the feasibility of tasks' priority inversions at run-time. Owing to such online priority inversion test, REORDER++ can generate highly randomized schedule of real-time tasks to mitigate the side-channel attack vulnerability. Experiments with synthesized task sets show that REORDER++ significantly outperforms the existing approaches in terms of schedule randomness.**

Manuscript received 6 September 2022; revised 5 February 2023; accepted 3 March 2023. Date of publication 10 March 2023; date of current version 25 October 2023. This work was supported in part by the National Key Research and Development Program of China under Grant 2021ZD0112400, in part by the National Natural Science Foundation of China under Grants 62072067, 62172069, 51978120, U1808206, 61602080, 61872052, 61906032, 61761136019, 61601080, 61602084, and 61772112, in part by the Dalian Young Star of Science and Technology Project under Grants 2021RQ055 and 2018RQ45, in part by the Social Science Foundation of Liaoning Province under Grant L17CTQ002, and in part by the Fundamental Research Funds for the Central Universities under Grants DUT21JC27 and DUT22LAB110. Recommended for acceptance by Dr. Sudip Misra. *(Corresponding author: Jiankang Ren.)*

Jiankang Ren, Zheng Wang, Hongrui Xie, Haihui Zhu, Chunxiao Liu, Kaiwen Wang, and Guozhen Tan are with the School of Computer Science and Technology, Dalian 116024, China (e-mail: rjk@dlut.edu.cn; zhengw@mail.dlut.edu.cn; 0511xhr@mail.dlut.edu.cn; 1017642109@mail.dlut.edu.cn; liuchunxiao@mail.dlut.edu.cn; 1572874923@mail.dlut.edu.cn; gztan@dlut.edu.cn).

Chi Lin is with the School of Software, Dalian University of Technology, Dalian 116024, China (e-mail: c.lin@dlut.edu.cn).

Mohammad S. Obaidat is with the Chair & Professor, Computer Science Department, and Director of Cybersecurity Center, University of Texas-Permian Basin, Odessa, TX 79762 USA, also with the King Abdullah II School of Information Technology, University of Jordan, Amman 11942, Jordan, also with the School of Computer and Communication Engineering, University of Science and Technology Beijing, Beijing 100083, China, and also with the Honorary Distinguished Professor, Amity University, Noida 201301, India (e-mail: msobaidat@gmail.com).

Digital Object Identifier 10.1109/TNSE.2023.3254653

***Index Terms*—Absolute busy interval analysis, embedded real-time systems, online priority inversion test, randomized real-time scheduling, side-channel attacks.**

I. Introduction

EMBEDDED real-time systems have been widely adopted in safety-critical fields such as aircraft, automobiles, medical devices and space vehicle [1], [2]. In order to deliver critical functionality, the execution of the real-time systems is generally predictable and deterministic. However, the deterministic execution pattern of the system is a double-edged sword which also leaves the system vulnerable to some external attacks [3], [4], [5], such as timing side-channel attack which is a security exploit based on information gained from the implementation of a system. For example, a schedule-based timing side-channel attack [6], [7] can exploit the deterministic execution pattern of real-time systems to obtain the system scheduling information. Once attackers have enough information about system internals, they are able to craft more effective attacks tailored to the particular system, such as destroying the stability of the system by injecting false data [8], [9] and damaging vehicle functions by covering control system signals [6], [10]. Note that, the embedded systems tend to be built from the common-off-the-shelf components to reduce the development costs, and to be remote monitoring and control over the WiFi, radio or cellular network to improve the efficiency and robustness of the system. Such openness of the embedded real-time systems increases the exposure to risks and vulnerabilities which provide many remote attack surfaces for attackers to launch side-channel attacks. For example, an unprivileged user-space program can be injected into the autonomous vehicle system through the external network, and then the route and the location of the vehicle can be predicted by using a prime-and-probe cache timing side-channel attack on the control software, without access to sensor inputs or protected state of control software [11], [12]. Consequently, it is utmost important to provide an efficient security strategy against side-channel attacks in real-time systems, such that both timing and security requirements can be ensured for the safety-critical systems.

As a critical ingredient for moving target defense (MTD) techniques [13], [14], schedule randomization is promising to reduce the amount of information obtained by the attacker through dynamically changing execution sequence of tasks in the scheduling process, and thus the timing side-channel attacks in

real-time systems can be mitigated [15], [16], [17]. The schedule randomization strategy for real-time systems can be generally classified into two categories [18]: offline randomized scheduling and online randomized scheduling. In offline randomized scheduling, the schedule is selected at random from a set of pre-generated schedules [18]. In contrast, in online randomized scheduling, the schedule is generated randomly at run-time [15], [17]. In this paper, we focus on the online randomized scheduling against timing side-channel attacks in dynamic-priority real-time systems. Note that, it is not trivial to efficiently generate feasible schedule at random, since the schedulability of the online generated schedule should be ensured.

For the dynamic-priority real-time systems, Chen et al. [17], [19] proposed a randomized scheduling method (named REORDER) to reduce determinism of adversary perception by offline analyzing the worst-case time budget for priority inversion of tasks, while ensuring the schedulability of the system. REORDER effectively obfuscates the deterministic timing behavior of such systems, especially under the condition of low and medium system loads. Nevertheless, REORDER suffers from the problem of pessimism in the offline calculation of tasks' time budget for priority inversion due to ignoring the run-time behavior and characteristics of the system, such as the changes of the priority and remaining execution time of each task at run-time. This makes the performance of REORDER degrades when serving the heavy-utilization tasks, and even cannot achieve randomized schedule for some heavy-utilization systems with few idle time. To address this problem, we propose an enhanced randomized scheduling strategy (named REORDER++) to increase the feasible opportunities of priority inversion by utilizing the run-time information of systems, such as the time budget for priority inversion and priority of each task at run-time, while ensuring the system schedulability. REORDER++ can enable more priority inversions during the tasks' scheduling to realize highly randomized schedule, this is because (i) the pessimism problem of tasks' time budget for priority inversion is effectively alleviated; (ii) when scheduling tasks at run-time, compared with the existing methods, we have more opportunities to schedule tasks.

Contributions: This paper makes the following contributions:

- We introduced an enhanced randomized scheduling strategy to break the deterministic task execution patterns of system by randomizing the task scheduling to mitigate the side-channel attack vulnerability, while guaranteeing the system schedulability;
- We proposed an online priority inversion test method to accurately analyze tasks' time budget for priority inversion through feasible absolute busy interval analysis to increase the feasible opportunities of priority inversion at run-time; and
- We evaluated the performance of REORDER++ with synthetic workloads, and compared it with the latest randomized scheduling method to validate its effectiveness and progressiveness.

Paper organization: The rest of the paper is structured as follows. Section II summarizes the related work of the works in this paper. Section III presents the system model, performs a problem description of timing side-channel attacks and introduces the background of the existing randomized scheduling method. In Section IV, we introduce the limitations of existing schedule randomization method, and propose an enhanced randomized scheduling strategy based on the combination of offline worst-case inversion budget analysis and online priority inversion test, and the absolute busy interval analysis method based on priority inversion area in details. We present our performance evaluation and compare our algorithm with the latest competing algorithm in Section V. Finally, we conclude this paper in Section VI.

II. Related Work

Researchers have demonstrated that the real-time scheduling in systems has become a source of information leakage, which motivated the development of defense mechanism of real-time system [6], [7], [20], [21], [22], [23]. Depending upon the time when the adversary launches the attack, the attacks on the real-time scheduling has been classified into four categories [24]: (1) posterior attack model: an attack is launched after the target task has completed its execution, such as replay attacks [25] and task parameters analysis [26], etc; (2) anterior attack model: an attack is mounted before the execution of the target task, such as false data injection attack [8], [27]; (3) pincer attack model: through combining the posterior and anterior attack model, the adversary analyzes the target task at load time and monitors its behavior after the target task has completed execution, such as schedule-based timing side-channel attacks [6], [7]; (4) concurrent attack model: an attack is performed while the target task is running, and it can be mounted by executing between the execution window of the target task's job. For example, the multi-target attack proved by Delimitrou et al. [28] and Nathuji et al. [29] can reduce the performance or quality of service of the attacked task.

As a typical pincer attack model in the real-time system, the schedule-based timing side-channel attack can steal the timing parameters from the task execution sequence by exploiting the predictability of real-time systems, and furthermore, obtain more scheduling information of the system [6], [26]. To defend against such attacks, some defense strategies have been proposed by introducing isolation mechanism and confusion mechanism in the schedule. Chen et al. [30] proposed a temporal isolation mechanism to protect against posterior scheduler side-channel attacks by preventing untrusted tasks from executing during specific time segments. However, this temporal isolation mechanism only plays a defensive effect against the posterior scheduler side-channel attacks and can not resist the threat posed by the schedule-based timing side-channel attack. In order to solve this problem, schedule randomization has been widen introduced as a confusion mechanism to strengthen the security mechanism in real-time systems. Yoon et al. [15] proposed a priority inversion based randomized scheduling method to resist such schedule-based timing side-channel attacks. In this method, the priority inversion is fulfilled by statically calculating the priority inversion budget, while achieving real-time guarantee. To further enhance the randomness of scheduling, Yoon et al. [16] utilizes

run-time information readily available at each scheduling decision point to increase the level of uncertainty in task schedules, while preserving the original schedulability. In order to reduce the certainty of time-triggered (TT) real-time systems, Krüger et al. [18], [31] proposed slot-level online randomization of schedules and offline schedule-diversification strategies. However, the schedulability analysis of all above methods mainly focuses on the fixed-priority scheduling policy, and cannot be directly applicable for dynamic-priority real-time systems. To obfuscate the predictable timing behavior of dynamic-priority real-time systems, Chen et al. [17], [19] proposed a priority inversion based randomized scheduling method (named $REORDER$). However, $REORDER$ suffers from the problem of pessimistic bounds of static analysis, since it calculates the worst-case inversion budget off-line while ignoring the run-time behavior and characteristics of the system.

III. System Model and Problem Description

A. System Model

The real-time system considered in this paper consists of N independent periodic real-time tasks, identified by $\Gamma = \{\tau_1, \ldots, \tau_N\}$, and they are scheduled on uniprocessor based on the preemptive dynamic-priority scheduling policy. Each task $\tau_i \in \Gamma$, $1 \leq i \leq N$, is characterized by a tuple (C_i, T_i, D_i), where

- C_i is the worst-case execution time (WCET) of task τ_i;
- T_i is the inter-arrival time (period) of task τ_i; and
- D_i is the relative deadline of task τ_i.

We assume that all tasks of Γ are implicit deadline tasks (i.e., $D_i = T_i$), and they are initially released simultaneously at time instant $t = 0$. The task τ_i can release a set of related jobs which are denoted as $\mathcal{J}_{i,k}$ that jointly provide some system functionalities. The release time and response time of $\mathcal{J}_{i,k}$ are denoted as $r_{i,k}$ and $R_{i,k}$, which belong to an infinite set $A(\tau_i) = \{0, T_i, 2T_i, \ldots\}$. For a job $\mathcal{J}_{i,k}$ of task τ_i released at $r_{i,k}$, its absolute deadline and finishing time are defined as $d_{i,k}$ and $f_{i,k}$, respectively. For each job $\mathcal{J}_{i,k}$ of each task $\tau_i \in \Gamma$, if $f_{i,k} - r_{i,k}$ is not larger than the deadline of task τ_i, the system is considered schedulable. For a job $\mathcal{J}_{i,k}$ of task $\tau_i \in \Gamma$, the lower-priority and higher-priority jobs of $\mathcal{J}_{i,k}$ are denoted by $lp(\mathcal{J}_{i,k})$ and $hp(\mathcal{J}_{i,k})$. For notational convenience, without causing any confusion, the job $\mathcal{J}_{i,k}$ of task τ_i is denoted as $\mathcal{J}_i$, and its release time, deadline, finishing time and response time are represented by r_i, d_i, f_i and R_i, respectively. In this paper, we assume a discrete notion of real-time $t = m\theta, m \geq 1$, where $\theta > 0$ is both the unit time and the smallest unit of preemption (called a slot). Since both task releases and scheduling activities occur at slot boundaries only, all timing values are specified as positive integers. We let H denote the hyperperiod of a task set, and it is calculated as the least common multiple of periods of all tasks in the task set.

B. Problem Description

Adversary Model: We consider an adversary that tries to launch a schedule-based timing side-channel attack to infer the scheduling information of real-time tasks, especially the arrival time and execution time of critical and periodic tasks in the system. Based on these obtained scheduling information, the adversary will be able to launch more attacks that vary with adversaries and the target systems. For example, the pulse-width modulation (PWM) task in real-time control system of smart cars controls the steering and throttle functions by periodically releasing its jobs to send out the PWM signals. An adversary can infer the execution time and future release time of the PWM task through the side-channel attack [6], thus overriding the PWM signals to control the steering and throttle functions of the smart car and threatening the safety of the victim system.

In this paper, we consider the timing side-channel attack for the dynamic-priority real-time systems provided in [7], [19], in which the adversary aims to infer the scheduling information in system. Our adversary model is based on the following assumptions:

1) The initial conditions at system start-up (e.g., the task's initial offset) and the information on all the tasks in the system may be not available for the adversary.
2) The adversary has ability to obtain some preknowledge of the system, such as the period of the tasks and the scheduling policy which is being used in system.
3) The adversary has the control of one user-space task or insert a non-critical task in the system without being discovered.
4) The adversary has the ability to access the system clock to read the system time.

Based on the above assumptions, the adversary has the following capabilities:

Capability 1: The adversary can use a system timer to measure and reconstruct the valid execution intervals of the controlled task, that have lower priority than the target task.

Capability 2: The adversary can analyze the execution intervals of the target task through the valid execution intervals of the controlled task.

By using the above capabilities, the adversary can effectively infer the scheduling information of the target task (denoted by τ_v) in system. An attacker controls an user-space task as an observer task (denoted by τ_o) and reconstructs the execution intervals between $[r_o, r_o + T_o - T_v]$ as the valid execution intervals. The valid execution intervals of τ_o are organized into the "schedule ladder diagram" – a timeline that is divided into windows that match the period of the target task. For the target task, its arrival time and execution time will be exposed through the effective preemption of τ_v to τ_o, and its future arrival time can also be exposed according to the period of the target task.

Example 1: Consider an observer task $\tau_o(C_o = 4, T_o = 10, D_o = 10)$ that is controlled by an adversary and a target task $\tau_v(C_v = 2, T_v = 8, D_v = 8)$ in system. Assume that the observer task has collected its execution intervals for a duration of $LCM(T_o, T_v)$ (i.e., 40 time units in this example) from $t = 41$. As shown in Fig. 1 , it is a simple schedule of tasks τ_o and τ_v using Earliest Deadline First (EDF) scheduling policy [32]. In EDF, the priorities of tasks are assigned according to the absolute deadline, the closer the deadline is, the higher the priority is. From Fig. 1, we can find that the reconstructed valid execution

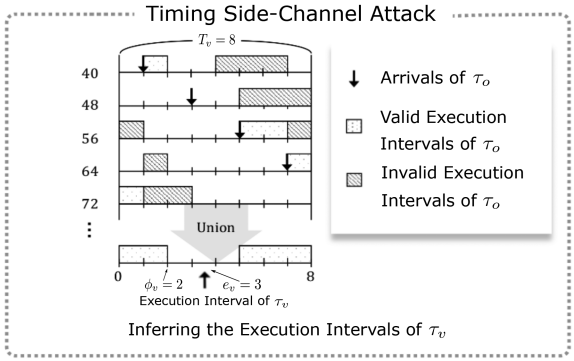
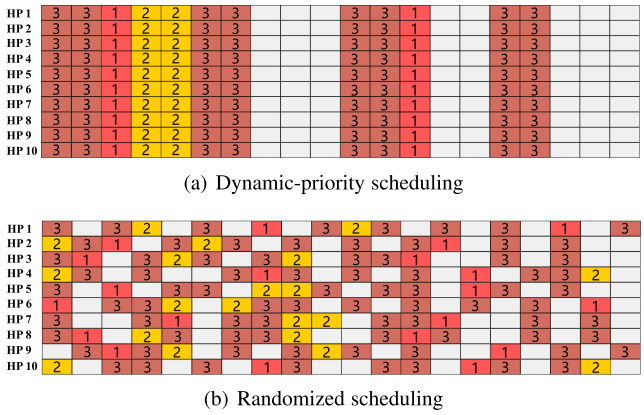


Fig. 1. A schedule-based timing side-channel attack from [7].



(a) Dynamic-priority scheduling

(b) Randomized scheduling

Fig. 2. Scheduling pattern of $\Gamma = \{\tau_1(C_1 = 1, T_1 = 10, D_1 = 10), \tau_2(2, 20, 20), \tau_3(2, 5, 5)\}$ scheduled by dynamic-priority scheduling and with scheduling randomization on an uniprocessor platform.

intervals of observer task τ_o are $\{[41, 42), [61, 63), [71, 73)\}$. By analyzing the valid execution intervals of τ_o, τ_v's execution intervals are exposed, and the arrival time ϕ_v and execution time e_v of τ_v can also be effectively inferred.

Defense Model: The key property of real-time systems is that the execution of periodic tasks is repeated with no or slight variations within periodic intervals (such as the hyperperiod of all tasks) to deliver critical functionality. Although such property is helpful to ensure the schedulability of the system, it also increases the success rate of the timing side-channel attack. Fig. 2 illustrates the scheduling pattern of $\Gamma = \{\tau_1(C_1 = 1, T_1 = 10, D_1 = 10), \tau_2(2, 20, 20), \tau_3(2, 5, 5)\}$ scheduled by dynamic-priority scheduling without scheduling randomization (Fig. 2(a)) and with scheduling randomization (Fig. 2(b)) on a uniprocessor platform. It shows the execution of the task in 200 time slots, and the number in each slot indicates the task index executed at the time slot. In Fig. 2(a), the scheduling pattern is repeated every 20 time units because the hyperperiod of tasks in Γ is 20. On the other hand, the scheduling pattern of task set Γ is randomized without schedulability loss (as shown in Fig. 2(b)). That means that every task both in Figs. 2(a) and (b) completes its execution without any deadline miss. The scheduling randomization breaks the deterministic and repetitive task scheduling pattern of real-time systems. Therefore, even if the adversary can record the exact scheduling of a certain hyperperiod, he can not launch an effective attack since the scheduling pattern it not repeated any more. This means that real-time scheduler can resist the timing side-channel attacks by randomizing its scheduling pattern. Thus, the following objective is expected to be achieved in this paper:

Goal: For a task set Γ that is schedulable under the preemptive EDF scheduling policy, the main objective of this paper is to randomize its schedule as much as possible, such that the chance of the adversary to launch a successful schedule-based timing side-channel attack can be mitigated, while guaranteeing the schedulability of the task set Γ.

IV. REORDER++ Randomized Scheduling Strategy

A. Motivation

To ensure the stability of the system, the real-time systems are often designed to be predictable and deterministic, that is, the real-time tasks in system are always be completed within the deadline, and will present repeated task schedule in different hyperperiods. However, the deterministic task execution pattern is also helpful for attackers to infer the inner scheduling information of the system through the external attacks [6], [33], such the timing side-channel attacks [6]. To prevent such attacks, we try to break the deterministic task execution pattern through scheduling randomization.

Before introducing our proposed randomized scheduling strategy REORDER++, we first briefly introduce the existing randomized scheduling strategy, then discuss the limitations of the existing work through an example of a timing side-channel attack that infers the arrival time and execution of a task in system, and motivate the proposed work in this paper. In [17], [19], a randomized scheduling strategy (called REORDER) is presented for dynamic-priority real-time system. REORDER realizes the randomized scheduling based on the analysis of tasks' worst-case priority inversion budget which is the maximum time budget that the job of the task can be blocked by lower priority jobs without causing any deadline miss. The basic idea of REORDER can be briefly described as below:

- For each task in the task set, the worst-case priority inversion budget is calculated offline.
- Based on the worst-case priority inversion budget of each task, a candidate job set is selected from the ready job set at each scheduling point.
- From the candidate job set, a job is randomly selected to execute to randomize the schedule.

It has been demonstrated that REORDER can randomize the schedule of real-time tasks compared to the traditional real-time scheduling approaches in dynamic-priority real-time systems. Nevertheless, for some heavy-utilization tasks, the performance of REORDER will decline and it cannot effectively generate randomized schedule to defend the timing side-channel attack. This is because REORDER ignores the run-time behavior and characteristics of the system and offline calculates the priority inversion budget by considering the priority inversion, which can happen every time. As illustrated in Fig. 3, a chain reaction may occur due to the arbitrary priority inversions, i.e., some

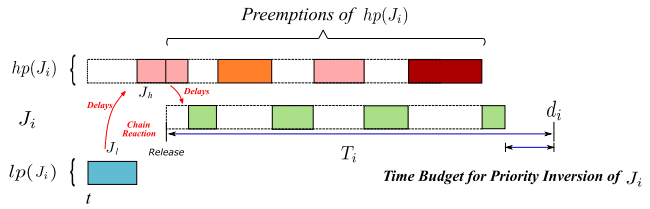


Fig. 3. Interference from $lp(\mathcal{J}_i)$ on $\mathcal{J}_i$ due to chain reaction.

TABLE I
A TASK SET FOR EXAMPLE 1

$Task$	C_i	$T_i = D_i$	V_i
τ_1	4	10	2
τ_2	1	20	−3
τ_3	1	4	0
τ_4	2	12	−2

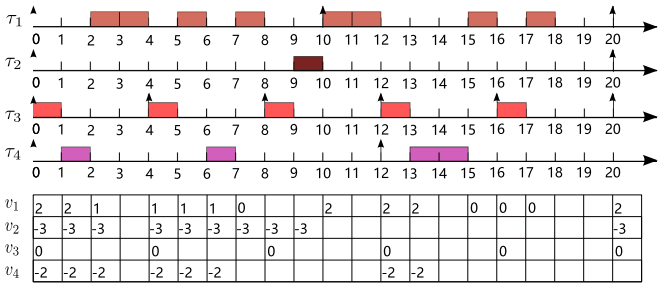


Fig. 4. Randomized schedule for Γ generated by REORDER.

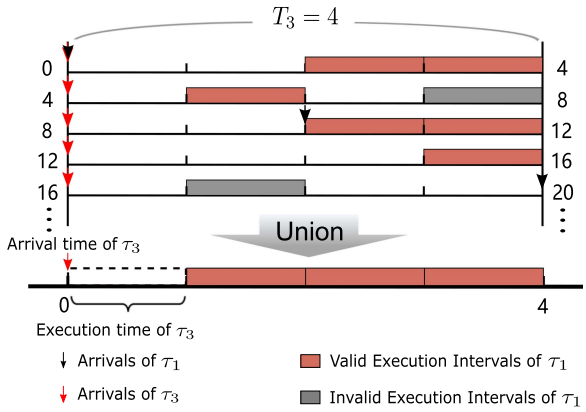


Fig. 5. Inferring the arrival time and execution time of τ_3 by timing side-channel attack.

lower-priority jobs in $lp(\mathcal{J}_i)$ delay the higher-priority job in $hp(\mathcal{J}_i)$, and then in turn delay $\mathcal{J}_i$. This chain reaction leads to the pessimistic value of time budget for priority inversion of tasks. All of them lead to the reduction of opportunity of priority inversion under the system schedulable constraint, and thus REORDER cannot effectively generate randomized schedule for the heavy-utilization tasks to defend the timing side-channel attack. In the following, we illustrate this problem with an example.

Example 2: Consider a task set $\Gamma = \{\tau_1, \tau_2, \tau_3, \tau_4\}$ with the parameters depicted in Table I scheduled under the EDF scheduling policy, where V_i is the off-line calculated worst-case priority inversion budget of REORDER. It is noteworthy that the priority inversion budgets of τ_2, τ_3 and τ_4 are no more than 0, which means the jobs of τ_2, τ_3 and τ_4 are not allowed be blocked by any of lower priority jobs. As shown in Fig. 4 , it is the randomized schedule generated by REORDER for the task set Γ and the change process of the priority inversion budget v_i of each task. From Fig. 4, we can find that the priority inversion budget v_i of $\mathcal{J}_i$ is initialized to V_i upon each activation of the jobs of τ_i. The jobs with $v_i > 0$ can be blocked by lower priority jobs, and v_i is decremented for each time unit. When v_i is decremented to 0, $\mathcal{J}_i$ is no longer allowed to be blocked by any lower priority jobs. As shown in Fig. 5 , a timing side-channel attack is launched to infer the arrival time and execution time of τ_3 in Γ. From Fig. 5, we can find that the arrival time and execution time of τ_3 can be inferred from the observation of τ_1's execution intervals by unioning the execution intervals of τ_1 into the scheduler ladders with width of $T_3 = 4$. The main reason for this is that any job of τ_3 is not allowed to be blocked by any of lower priority jobs since each job of τ_3 does not have a positive priority inversion budget (i.e., $v_3 = 0$).

In order to address the limitation of REORDER, we propose an enhanced randomized scheduling strategy, named REORDER++, based on analysis of priority inversion budget and the online priority inversion test, to counteract the timing side-channel attacks in dynamic-priority real-time systems. On the basis of online priority inversion test, the feasible opportunities of priority inversion can be increased by considering the run-time information of the system. Thus, highly diverse feasible temporal execution schedule of tasks can be generated for the heavy-utilization dynamic-priority real-time systems with REORDER++ to mitigate the side-channel attack vulnerability.

B. Overview of REORDER++

The key idea of REORDER++ is to counteract the timing side-channel attacks by random priority inversion with the online priority inversion test. However, it is a non-trivial task because we need to guarantee the safety of the randomness introduced into the schedule. Before presenting the details of REORDER++, we first give its overview to demonstrate how it works. REORDER++ can be divided into the offline processing and the online processing. In the offline processing, the worst-case priority inversion budget is calculated based on the response time analysis of EDF [17]. In the online processing, it includes the conservative candidate job set generation and online randomized scheduling which are presented as follows:

Step 1: Conservative candidate job set generation:

Based on the worst-case priority inversion budget of each task, the conservative candidate job set L_C^t is generated from the ready job set R_Q^t at scheduling point t by the following steps, which include jobs that can be randomly selected while ensuring the system schedulability.

1) The priority inversion budget v_i of each job $\mathcal{J}_i$ in ready job set R_Q^t at time t is calculated by the following rules: (i) when the job of τ_i is released, v_i is initialized to V_i;

(ii) when $\mathcal{J}_i$ is blocked by any lower priority jobs, v_i is reduced for each time unit until it is reduced to 0.

2) We check the jobs in R_Q^t in descending order of priority, and the jobs $\mathcal{J}_i$ with $v_i > 0$ are put into L_C^t in turn, until a job $\mathcal{J}_{last}$ with $v_{last} \leq 0$ is checked. Note that, the job $\mathcal{J}_{last}$ is also put into L_C^t, since the priority inversion of $\mathcal{J}_{last}$ will not affect the system schedulability.

Step 2: Online randomized scheduling: A job $\mathcal{J}_i$ is randomly selected from the ready job set R_Q^t and is performed by the following cases:

1) If $\mathcal{J}_i \in L_C^t$, the job $\mathcal{J}_i$ is executed for ω_i^t time units at time t. Note that, to ensuring the schedulability of system, the execution time ω_i^t of $\mathcal{J}_i$ is no more than $\min\{\widehat{C}_i^t, \widehat{v}\}$, where $\widehat{C}_i^t$ is the remaining execution time of $\mathcal{J}_i$ at time t and $\widehat{v} = \min\{v_j \mid \mathcal{J}_j \in R_Q^t \wedge d_j < d_i\}$ is the minimum priority inversion budget of the higher-priority job $hp(\mathcal{J}_i)$ in ready job set R_Q^t.
2) If $\mathcal{J}_i \notin L_C^t$, an online priority inversion test with size ω_i^t is performed for $\mathcal{J}_i$ to judge whether it can be executed at time t. If $\mathcal{J}_i$ passes the priority inversion test, it will be executed for ω_i^t time units, otherwise, a new job $\mathcal{J}_i$ is randomly reselected from L_C^t and is performed by the Case (1). Note that, the priority inversion test size ω_i^t of $\mathcal{J}_i$ is no more than $\widehat{C}_i^t$, where $\widehat{C}_i^t$ is the remaining execution time of $\mathcal{J}_i$ at time t.

The next scheduling decision will be made at:

$$t' = t + \omega_i^t. \tag{1}$$

and, REORDER++ iterates Step 1 and Step 2 to realize the randomized schedule.

To further increase the randomness of the schedule, we introduce the *idle time scheduling* by treating the idle time in the system as an additional idle task τ_{idle} in Γ to participate in online randomized scheduling [15], [17]. The idle task τ_{idle} has its unique properties (a) the lowest-priority, (b) infinite period and deadline and (c) infinite execution time. For the extended task set $\Gamma' = \Gamma \cup \{\tau_{idle}\}$, REORDER++ can disturb the deterministic scheduling of idle time due to the work-conserving nature of real-time scheduling algorithm. In addition, to increase more randomness in the execution intervals, we introduce the *fine-grained switching* which allows the scheduling to be interrupted at a random time even if the job's continued execution will not affect the schedulability of the task set. The fine-grained switching is enforced by setting the job's execution time ω_i^t to a random value, i.e., $\omega_i^t = rand(1, \omega_i^t)$. In REORDER++, we take *idle time scheduling* and *fine-grained switching* as two addition modules to increase the uncertainty in the schedule.

It is worth mentioning that the online priority inversion test is challenging since the schedulability of the system should be preserved. In the following, we will provide the process of the online priority inversion test in details.

C. Online Priority Inversion Test

In this section, we first introduce the main process of priority inversion test in details. Then, we give the schedulable condition about the absolute busy interval analysis for the priority inversion test. To accurately analysis the time scope which could be delayed by the priority inversion, we give the priority inversion area, which only the jobs in it can be delayed by the priority inversion, and we propose an efficient online priority inversion test algorithm.

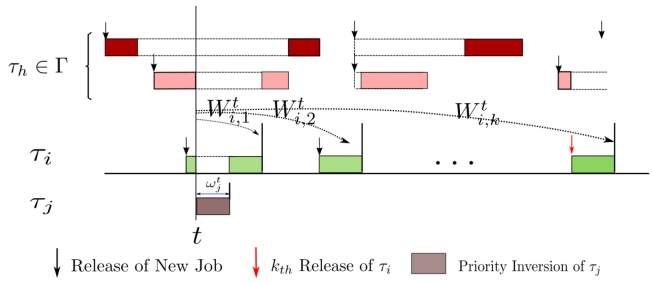


Fig. 6. The absolute busy interval $W_{i,k}^t$ of $\mathcal{J}_{i,k}$.

To illustrate the process of the online priority inversion test, we suppose that a scheduling decision is made at time t, and the selected job $\mathcal{J}_j \notin L_C^t$. During the priority inversion test, we perform a simulation that the job $\mathcal{J}_j$ executes for ω_j^t time units at time t by priority inversion and the higher relative priority jobs $hp(\mathcal{J}_j)$ is delayed ω_j^t time units accordingly.

To ensure the schedulability of the task set, an absolute busy interval analysis is enforced to judge whether $\mathcal{J}_j$ can be executed for ω_j^t time units at time t. If the priority inversion of $\mathcal{J}_j$ will affect the schedulability of task set, $\mathcal{J}_j$ fails the priority inversion test and a new job will be reselected from the conservative candidate job set L_C^t. Since priority inversion may indirectly affect the release of future jobs, each job of tasks in Γ should be enforced an job-level schedulability analysis to judge whether it will miss its deadline. To perform the job-level schedulability analysis for each job $\mathcal{J}_{i,k}$ of each task τ_i in Γ, we define the absolute busy interval, which includes the amount of execution time of $\mathcal{J}_{i,k}$ itself, and interference from preemptions of the higher priority jobs $hp(\mathcal{J}_{i,k})$ and priority inversion of $\mathcal{J}_j$ after time t.

Definition 4.1: Given a job $\mathcal{J}_j$ of τ_j that is executed by priority inversion at time t and a job $\mathcal{J}_{i,k}$ of task τ_i, the ***absolute busy interval*** of $\mathcal{J}_{i,k}$ with respect to time t, denoted by $W_{i,k}^t$, is defined as:

$$W_{i,k}^t = \omega_j^t + \sum_{O_h^t \leq d_{i,k}} \left(\widehat{C}_h^t + \left\lfloor \frac{d_{i,k} - O_h^t}{T_h} \right\rfloor * C_h \right). \tag{2}$$

where ω_j^t is the priority inversion size of job $\mathcal{J}_j$ at time t, $d_{i,k}$ is the absolute deadline of job $\mathcal{J}_{i,k}$, O_h^t is the first release time of job $\mathcal{J}_h$ after time t, and $\widehat{C}_h^t$ is the remaining execution time of job $\mathcal{J}_h$ with $O_h^t \leq d_{i,k}$ after time t and before time O_h^t.

For a job $\mathcal{J}_h$ with $O_h^t \leq d_{i,k}$ of τ_h, its first release time O_h^t after time t can be calculated as follows:

$$O_h^t = \max\left\{ \left\lceil \frac{t}{T_h} \right\rceil, \left\lfloor \frac{t}{T_h} \right\rfloor + 1 \right\} * T_h. \tag{3}$$

As shown in Fig. 6, it is the illustration for the calculation of the absolute busy interval $W_{i,t}^k$ of the kth job $\mathcal{J}_{i,k}$ of task τ_i after

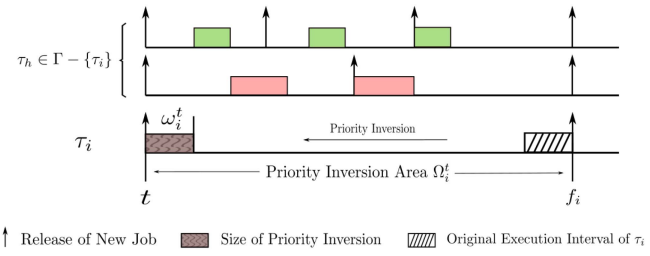


Fig. 7. The priority inversion area Ω_i^t of $\mathcal{J}_i$.

t. We can find that $W_{i,k}^t$ includes the priority inversion with size ω_j^t of job $\mathcal{J}_j$, the execution time of $\mathcal{J}_{i,k}$ and the execution time of $\mathcal{J}_h$ with $O_h^t \le d_{i,k}$.

To ensure the system schedulability under the priority inversion, we give the schedulable condition about the absolute busy interval as following Lemma:

Lemma 4.1: Let $\Gamma = \{\tau_1, \tau_2, \ldots, \tau_N\}$ be a schedulable task set on uniprocessor under the EDF policy. For any hyperperiod mH ($m \ge 1$), at any time $(m-1)H \le t \le mH$, the job $\mathcal{J}_j$ of task $\tau_j \in \Gamma \cup \{\tau_{idle}\}$ is executed by priority inversion at time t, the following equation is satisfied for each job $\mathcal{J}_{i,k}$ of each task $\tau_i \in \Gamma$,

$$t + W_{i,k}^t \le d_{i,k}. \tag{4}$$

Then the task set Γ is also schedulable under the priority inversion with size ω_j^t by the job $\mathcal{J}_j$ at time t.

Proof: Suppose the task set Γ is unschedulable for the priority inversion at time t when (4) is satisfied. Since only the schedulability of the ready jobs after t is affected by the priority inversion at t, there must be a job $\mathcal{J}_{i,k}$ which misses its deadline after t (i.e., $t + R_{i,k} > d_{i,k} > t$). Note that, the schedulability of idle task does not need to be considered, because its deadline is infinite. Based on (2), the absolute busy interval $W_{i,k}^t$ of $\mathcal{J}_{i,k}$ includes the priority inversion of $\mathcal{J}_j$, interference of $hp(\mathcal{J}_{i,k})$ and execution time of $\mathcal{J}_{i,k}$ itself. For the job $\mathcal{J}_{i,k'}$, $d_{i,k'} < d_j$, there is no redundant part in the calculation of $W_{i,k'}^t$, thus $W_{i,k'}^t = R_{i,k'}$. For the job $\mathcal{J}_{i,k'}$, $d_{i,k'} \ge d_j$, since $\mathcal{J}_j \in hp(\mathcal{J}_{i,k'})$ and ω_j^t is the execution time of $\mathcal{J}_j$, thus the ω_j^t is calculated repeatedly in $W_{i,k'}$, which leads to $W_{i,k'}^t > R_{i,k'}$. We can obtain that, for the job $\mathcal{J}_{i,k}$, its absolute busy interval $W_{i,k}^t \ge R_{i,k}$, hence $t + W_{i,k}^t > d_{i,k}$. This is inconsistent with the satisfiability of (4). Therefore, the task set Γ is also schedulable under the priority inversion with size ω_j^t by the job $\mathcal{J}_j \in lp(\mathcal{J}_i)$ at time t.

According to Lemma 4.1, in order to check the feasibility of a priority inversion, we need to calculate the absolute busy interval of all jobs of all tasks in Γ released in a hyperperiod. However, in fact, it is not necessary to compute all absolute busy intervals of all jobs released in the hyperperiod, but only those jobs in a time scope which only the jobs in it can be delayed by the priority inversion. As shown in Fig. 7, we illustrate a priority inversion area Ω_i^t where only the ready jobs in this area will be delayed by the priority inversion of job $\mathcal{J}_i \in \tau_i$ at time t. Now, we fist give the formal definition of the priority inversion area.

Definition 4.2: Given a job $\mathcal{J}_i$ of task $\tau_i \in \Gamma \cup \{\tau_{idle}\}$ that is executed at time t by priority inversion with size ω_i^t, the ***priority inversion area*** of $\mathcal{J}_i$, denoted by Ω_i^t, is defined as the time scope in schedule, which could be delayed by the priority inversion of $\mathcal{J}_i$ at time t.

Lemma 4.2: Let $\Gamma = \{\tau_1, \tau_2, \ldots, \tau_N\}$ be a schedulable task set on uniprocessor under the EDF policy. Given a job $\mathcal{J}_i$ of task $\tau_i \in \Gamma \cup \{\tau_{idle}\}$ that is executed by priority inversion at time t. The range of priority inversion area Ω_i^t of $\mathcal{J}_i$ is $[t, f_i]$ and only the execution of tasks in priority inversion area Ω_i^t can be delayed by the priority inversion of $\mathcal{J}_i$.

Proof: The priority inversion of $\mathcal{J}_i$ of τ_i at time t is to advance the execution interval with size ω_i^t of $\mathcal{J}_i$ to time t, which the execution interval is originally finished before time f_i. According to the work-conserving nature of real-time scheduling algorithm, the execution of jobs after time t will not be delayed more than ω_i^t time units by the priority inversion. Thus, the response time of jobs after f_i will not be increased, which means the execution of jobs after f_i will not be delayed by the priority inversion of $\mathcal{J}_i$ at time t. Therefore, the range of priority inversion area Ω_i^t of $\mathcal{J}_i$ is $[t, f_i]$ and only the execution of tasks in priority inversion area Ω_i^t can be delayed by the priority inversion of $\mathcal{J}_i$.

Since only the scheduliability of the ready jobs in the priority inversion area are affected by the priority inversion, based on Lemmas 4.1 and 4.2, we can obtain the following Lemma for the schedulable condition about the absolute busy interval analysis on the priority inversion area.

Lemma 4.3: Let $\Gamma = \{\tau_1, \tau_2, \ldots, \tau_N\}$ be a schedulable task set on uniprocessor under the EDF policy. For any hyperperiod mH ($m \ge 1$), at any time $(m-1)H \le t \le mH$, the job $\mathcal{J}_j$ of task $\tau_j \in \Gamma \cup \{\tau_{idle}\}$ that is executed by priority inversion at time t, the following equation is satisfied for any job $\mathcal{J}_{i,k}$ in the priority inversion Ω_j^t,

$$t + W_{i,k}^t \le d_{i,k}. \tag{5}$$

Then the task set Γ is also schedulable under the priority inversion with size ω_j^t by the job $\mathcal{J}_j$ at time t.

Proof: Suppose the task set Γ is unschedulable for the priority inversion at time t when (5) is satisfied. Based on Lemma 4.2, only the schedule of the jobs in the priority inversion area Ω_j^t are affected by the priority inversion at time t, thus, we consider such job $\mathcal{J}_{i,k}$ in Ω_j^t missed its deadline (i.e., $t + R_{i,k} > d_{i,k}$). Based on (5) and Lemma 4.1, We can obtain that $W_{i,k}^t \ge R_{i,k}$, hence $t + W_{i,k}^t > d_{i,k}$. This is inconsistent with the satisfiability of (5). Therefore, the task set Γ is also schedulable under the priority inversion with size ω_j^t by the job $\mathcal{J}_j$ at time t.

Based on Lemma 4.3, we proposed an online priority inversion test algorithm. As shown in Algorithm 1, it is the algorithm written in pseudocode. In the algorithm, the schedulability of jobs released in the priority inversion area is analyzed based on the absolute busy interval (Lines 1-10). Each job $\mathcal{J}_{h,k}$ of each task $\tau_h \in \Gamma - \{\tau_i\}$ is considered to perform the absolute busy interval analysis (Line 1). To determine the scope of the analysis, we take the absolute deadline of the last job released in the priority inversion area as the upper bound, and calculate the absolute busy interval of $\mathcal{J}_{h,k}$ (Lines 2-4). If any job in the priority inversion area misses its deadline, this means $\mathcal{J}_i$ is not allowed to execute at time t and the test of $\mathcal{J}_i$ is stopped

Algorithm 1: Online priority inversion test algorithm.

Input: The tested job $\mathcal{J}_i$, the task set Γ for schedulability analysis, priority inversion size ω_i^t, the scheduling point t.
Output: $true$ or $false$.

1: **for** each job $\mathcal{J}_h$ of task $\tau_h \in \Gamma - \{\tau_i\}$ **do**
2: $d_{h,0} \leftarrow (\lfloor \frac{t}{T_h} \rfloor + 1) * T_h$
3: **while** $d_{h,k} <= \left\lceil \frac{(\lfloor \frac{t}{T_i} \rfloor + 1) * T_i}{T_h} \right\rceil * T_h$ **do**
4: Calculate $W_{h,k}^t$ using (2).
5: **if** $t + W_{h,k}^t > d_{h,k}$ **then**
6: **return** $false$
7: **end if**
8: $d_{h,k} \leftarrow d_{h,k} + T_h$
9: **end while**
10: **end for**
11: // All jobs of τ_h are schedulable.
12: **return** $true$

(Lines 5-7). To perform absolute busy interval analysis for $\mathcal{J}_{h,k}$, its absolute deadline is iteratively increased by the period T_h of task τ_h (Lines 8). If all jobs of all tasks $\tau_h \in \Gamma - \{\tau_i\}$ are schedulable, which means the job $\mathcal{J}_i$ passes the priority inversion test (Lines 11–12).

From Algorithm 1, we can find that the time complexity of the online priority inversion test for job $\mathcal{J}_i$ mainly depends on the absolute busy interval analysis and the number of jobs in the priority inversion area Ω_i^t. Thus, the time complexity of priority inversion test for job $\mathcal{J}_i$ is $O(N \cdot \alpha_i)$, where N is the number of jobs in the ready job set R_Q^t at time t and $\alpha_i = \sum_{\mathcal{J}_h \in \Omega_i^t} \left\lceil \frac{f_i - t}{T_h} \right\rceil$ which is the number of jobs in the priority inversion area Ω_i^t.

D. Summary of REORDER++

In this section, we summarize the REORDER++ randomized scheduling strategy. Based on the online priority inversion test, we propose an enhanced randomized scheduling algorithm. By exploring the possibility of priority inversion for each job in the ready job set at each scheduling point, we can reasonably perform a priority inversion at the job-level, so as to achieve a high-degree schedule randomization. In the following, we introduce the REORDER++ algorithm.

Algorithm 2 is the REORDER++ algorithm written in pseudocode. The ready job set R_Q^t is sorted in descending order of priority in the current scheduling point t. The algorithm starts with the process of conservative candidate job set generation(Line 1). The jobs $\mathcal{J}_k \in R_Q^t$ with $v_k > 0$ are put into L_C^t in descending order of priority until a job $\mathcal{J}_k$ with $v_k \leq 0$ is checked and put into L_C^t(Lines 2-10). Then the online random scheduling process starts by setting the variables $flag$ and ω_i^t to $false$ and 0, respectively(Lines 11-13). The job $\mathcal{J}_i$, which is randomly selected from the ready job set $R_Q^t \cup \mathcal{J}_{idle}$ at time t (Line 14), is judged to be $\mathcal{J}_i \in L_C^t$ or not. If the job $\mathcal{J}_i \in L_C^t$, the execution time ω_i^t of $\mathcal{J}_i$ is set to $\min\{\widehat{C}_i^t, \widehat{v}\}$ and $\mathcal{J}_i$ can be executed for ω_i^t time units directly (Line 15-17). If the job $\mathcal{J}_i \notin L_C^t$, the size of priority inversion ω_i^t is set to $rand(1, \widehat{C}_i^t)$ and a priority

Algorithm 2: REORDER++ algorithm.

Input: The current scheduling point t, the task set Γ, the ready job set R_Q^t sorted by descending priority.
Output: Job $\mathcal{J}_i$ that can be executed and next scheduling point t'.

1: /* Conservative candidate job set generation */
2: $k \leftarrow 1$
3: **while** $\mathcal{J}_k \in R_Q^t$ && $k \leq R_Q^t.size()$ **do**
4: **if** $v_k \leq 0$ **then**
5: $L_C^t \leftarrow L_C^t \cup \{\mathcal{J}_k\}$
6: break
7: **end if**
8: $L_C^t \leftarrow L_C^t \cup \{\mathcal{J}_k\}$
9: $k \leftarrow k + 1$
10: **end while**
11: /* Online random scheduling */
12: $flag \leftarrow false$.
13: $\omega_i^t \leftarrow 0$.
14: Randomly select $\mathcal{J}_i \in R_Q^t \cup \{\mathcal{J}_{idle}\}$.
15: **if** $\mathcal{J}_i \in L_C^t$ **then**
16: $\omega_i^t \leftarrow min\{\widehat{C}_i^t, \widehat{v}\}$
17: **end if**
18: **if** $\mathcal{J}_i \notin L_C^t$ **then**
19: $\omega_i^t \leftarrow rand(1, \widehat{C}_i^t)$
20: $flag \leftarrow$ PriorityInversionTest($\mathcal{J}_i$, Γ, ω_i^t, t)
21: **if** !$flag$ **then**
22: Randomly reselect $\mathcal{J}_i$ from L_C^t
23: $\omega_i^t \leftarrow min\{\widehat{C}_i^t, \widehat{v}\}$
24: **end if**
25: **end if**
26: $\omega_i^t \leftarrow rand(1, \omega_i^t)$
27: **for** each job $\mathcal{J}_h \in R_Q^t$ **do**
28: **if** $d_h < d_i$ && $v_h > 0$ **then**
29: $v_h \leftarrow v_h - \omega_i^t$
30: **end if**
31: **end for**
32: $t' \leftarrow t + \omega_i^t$
33: // The next scheduling decision will be made at t'.
34: **return** $(\mathcal{J}_i, t')$

inversion test with size ω_i^t is enforced for $\mathcal{J}_i$ (Lines 18-20). If $\mathcal{J}_i$ fails the priority inversion test, a new job is randomly reselected from L_C^t and reset its execution time ω_i^t (Lines 21-25). If the selected job $\mathcal{J}_i$ passes the priority inversion budget or $\mathcal{J}_i \in L_C^t$ then $\mathcal{J}_i$ is executed for $rand(1, \omega_i^t)$ time units at time t and the priority inversion budget of jobs $\mathcal{J}_h \in hp(\mathcal{J}_i)$ is reduced accordingly(Lines 26-31). Then the next scheduling decision will be made at time $t' = t + \omega_i^t$ (Lines 32).

In the process of REORDER++ algorithm, a job $\mathcal{J}_i$ is randomly selected from the ready job set R_Q^t. If $\mathcal{J}_i \in L_C^t$, the time complexity of REORDER++ algorithm is $O(N)$, where N is the number of jobs in R_Q^t. If $\mathcal{J}_i \notin L_C^t$, the time complexity of REORDER++ algorithm is $O(N \cdot \alpha_i)$, where $\alpha_i = \sum_{\mathcal{J}_h \in \Omega_i^t} \left\lceil \frac{f_i - t}{T_h} \right\rceil$ which is the number of jobs in the priority inversion area Ω_i^t.

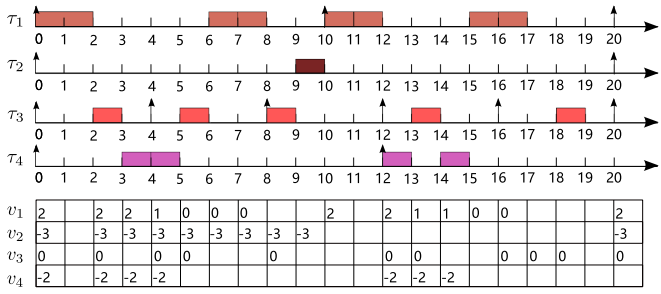


Fig. 8. Randomized schedule for Γ generated by REORDER++.

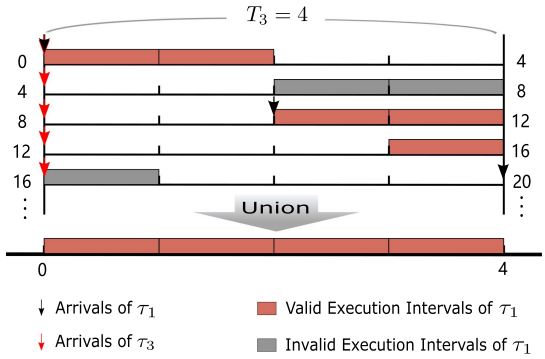


Fig. 9. Timing side-channel attack fails to infer the arrival time and execution time of τ_3 by observing the execution of τ_1.

Now, we briefly discuss the schedulability of our proposed REORDER++ algorithm. For each task set Γ, the priorities of all tasks allocated to it are assigned according to the EDF policy. We prove that the priority inversion implemented by REORDER++ algorithm always ensures that the task set Γ is schedulable.

Theorem 4.1: Let $\Gamma = \{\tau_1, \tau_2, \ldots, \tau_N\}$ be a schedulable task set on uniprocessor under the EDF policy, then it is schedulable with the REORDER++ randomized scheduling strategy described above.

Proof: The theorem is an immediate application of Lemma 4.3. Each job in R_Q^t selected by REORDER++ strategy falls into either of the following cases:

1) For any $\mathcal{J}_i$, if $\mathcal{J}_i \in L_C^t$, then it satisfies schedulability analysis in REORDER. Hence, task set Γ is schedulable.
2) For any $\mathcal{J}_i$, if $\mathcal{J}_i \notin L_C^t$, then it should pass the priority inversion test. According to Lemma 4.2, the task set Γ is schedulable under the priority inversion with size ω_i^t by the job $\mathcal{J}_i$. Hence, task set Γ is schedulable.

Therefore, the task set Γ that is schedulable under the EDF policy is also schedulable under the REORDER++ randomized scheduling strategy.

Example 3: Consider the task set $\Gamma = \{\tau_1, \tau_2, \tau_3, \tau_4\}$ with parameters depicted in Table I again. As shown in Fig. 8 , it is the randomized schedule generated by REORDER++ for the task set Γ and the change process of the priority inversion budget v_i of each task. From Fig. 8, we can find that the jobs of tasks in Γ (e.g., τ_2, τ_3 and τ_4) with non-positive priority inversion budget v_i can be blocked by lower priority jobs and will not cause any deadline miss. The following skips show how the randomized schedule generated by REORDER++.

- At $t = 0$, the ready job set $R_Q^0 = \{\mathcal{J}_1, \mathcal{J}_2, \mathcal{J}_3, \mathcal{J}_4\}$, the conservative candidate job set $L_C^0 = \{\mathcal{J}_3\}$. $\mathcal{J}_1$ is randomly selected. Since $\mathcal{J}_1 \notin L_C^0$, and $\mathcal{J}_1$ can passes the priority inversion test with size $\omega_1^0 = 2$, thus, the job $\mathcal{J}_1$ is executed for $\omega_1^0 = 2$ time units.
- At $t = 2$, $R_Q^2 = \{\mathcal{J}_1, \mathcal{J}_2, \mathcal{J}_3, \mathcal{J}_4\}$ and $L_C^2 = \{\mathcal{J}_3\}$. $\mathcal{J}_3$ is randomly selected. Since $\mathcal{J}_3 \in L_C^2$, the job $\mathcal{J}_3$ is directly executed for $\omega_3^2 = 1$ time unit.
- At $t = 3$, $R_Q^3 = \{\mathcal{J}_1, \mathcal{J}_2, \mathcal{J}_4\}$ and $L_C^3 = \{\mathcal{J}_1, \mathcal{J}_4\}$. $\mathcal{J}_4$ is randomly selected. Since $\mathcal{J}_4 \in L_C^3$, the job $\mathcal{J}_4$ is directly executed for $\omega_4^3 = 1$ time unit.
- At $t = 4$, $R_Q^4 = \{\mathcal{J}_1, \mathcal{J}_2, \mathcal{J}_3, \mathcal{J}_4\}$ and $L_C^4 = \{\mathcal{J}_3\}$. $\mathcal{J}_4$ is randomly selected. Since $\mathcal{J}_4 \notin L_C^4$, and $\mathcal{J}_4$ can pass the priority inversion test with size $\omega_4^4 = 1$. Thus, the job $\mathcal{J}_4$ is executed for $\omega_4^4 = 1$ time unit. The next scheduling decision will be made at $t = 5$.
- Going further along, at $t = 17$, $R_Q^{17} = \{\mathcal{J}_3\}$ and $L_C^{17} = \{\mathcal{J}_3\}$. $\mathcal{J}_3$ is randomly selected. Since $\mathcal{J}_3 \in L_C^{17}$, and the job $\mathcal{J}_3$ is directly executed for $\omega_3^{17} = 1$ time unit.
- From $t = 18$ to $t = 20$, there is no newly arrived job, so the processor is back to idle state.

As shown in Fig. 9 , a same timing side-channel attack as in Example 1 is launched to infer the arrival time and execution time of τ_3 in Γ. From Fig. 9, we can find that the arrival time and execution time of τ_3 cannot be inferred from the observation of τ_1's execution intervals by timing side-channel attack. This is because the schedule of τ_3 is randomized by REORDER++.

V. Performance Evaluation

In this section, we experimentally evaluate the performance of the REORDER++ randomized scheduling strategy proposed in this paper in terms of schedule entropy and compare it with the existing competing scheduling approaches. In the experiment, we implemented the following scheduling algorithms with different schemes:

- Vanilla EDF: A dynamic priority scheduling algorithm which assigns priorities to jobs of tasks according to their absolute deadlines in a dynamic fashion;
- REORDER (Base): A randomized scheduling strategy which randomizes the schedule for original task set Γ from [17], [19];
- REORDER (IT): A randomized scheduling strategy which obfuscates the EDF scheduling policy with idle time scheduling for augmented task set (i.e, $\Gamma \cup \{\tau_{idle}\}$) from [17], [19];
- REORDER (FT): A randomized scheduling strategy which obfuscates the EDF scheduling policy with idle time scheduling and fine-grained switching for augmented task set (i.e, $\Gamma \cup \{\tau_{idle}\}$) from [17], [19];
- REORDER++(Base): Our randomized scheduling strategy which randomizes the schedule for original task set Γ;
- REORDER++(IT): Our randomized scheduling strategy with idle time scheduling for augmented task set (i.e, $\Gamma \cup \{\tau_{idle}\}$); and

- REORDER++(FT): Our randomized scheduling strategy with idle time scheduling and fine-grained switching for augmented task set (i.e, $\Gamma \cup \{\tau_{idle}\}$).

A. Evaluation Metric

We take the schedule entropy proposed in [17] as the metric to measure the uncertainty of schedule and take the inference precision and inference success rate proposed in [19] as the metric to measure the defense capability in this evaluation. in this evaluation. For two schedule sequences with the same length, higher schedule entropy implies more randomness are introduced in the given schedule. Note that, in order to intuitively represent the scheduling entropy of K hyperperiods, we do not adopt the normalized value, which is different from [17]. To evaluate the defense capability, we introduce DyPs – a timing side-channel attack algorithm for dynamic priority real-time systems proposed in [19]. In the attack phase of DyPs, we are concerned with how close the inferred initial offset $\widetilde{\phi_v}$ is to the actual initial offset ϕ_v of the attacked task τ_v and $\widetilde{\phi_v}$ can be on either side of ϕ_v. The closer $\widetilde{\phi_v}$ and ϕ_v are, the higher inference precision is, and $\widetilde{\phi_v} = \phi_v$ means the inference is successful.

Schedule Entropy: Consider $K > 1$ hyperperiods for task set Γ with hyperperiod-length H, the schedule entropy, denoted by $\mathcal{H}(\widehat{C}_t^k, \pi, m, K)$, is defined as,

$$\mathcal{H}(\widehat{C}_t^k, \pi, m, K) = -\frac{1}{K}\sum_{k=1}^{K}\sum_{t=0}^{H-1} log_2 \widehat{C}_t^k. \tag{6}$$

where $\widehat{C}_t^k = \frac{1}{K}|\{k' : \delta(X_t^k(m), X_t^{k'}(m)) \leq \pi, 1 \leq k' \leq K\}|$, which denotes the dissimilarity of two scheduling time series X_t^k and $X_t^{k'}$, π is a given dissimilarity threshold and m is a given interval length of time series, $1 \leq m \leq H$. $\widehat{C}_t^k$ can be calculated by Hamming distance [34]. For two given vectors $U = [u_i]_{1\leq i\leq m}$ and $V = [v_i]_{1\leq i\leq m}$ of size m, $\delta(U, V) = \sum_{i=1}^{m}\prod(u_i \neq vi)$, where $\prod(u_i \neq vi)$ equals 1 if $u_i \neq vi$ or 0 other wise. In the experiment, we observed the schedule for $K = 100$ hyperperiods for each task set.

Inference Precision: The inference precision of ϕ_v for attacked task τ_v, denoted by Π_v^o, is defined as,

$$\Pi_v^o = \left|\frac{\epsilon}{T_v/2} - 1\right|. \tag{7}$$

where $\epsilon = |\widetilde{\phi_v} - \phi_v|$. The value of Π_v^o is a real number in the range $0 < \Pi_v^o \leq 1$. A larger Π_v^o indicates that the inference $\widetilde{\phi_v}$ is more precise in inferring ϕ_v.

Inference Success Rate: The inference success rate is defined as the percentage of the tested task sets in which the inferences are successful for a given test condition, where the inferences are successful means the attacker can exactly infer the initial offset of attacked task τ_v (i.e, $\widetilde{\phi_v} = \phi_v$), otherwise the inference fails.

B. Evaluation Setup

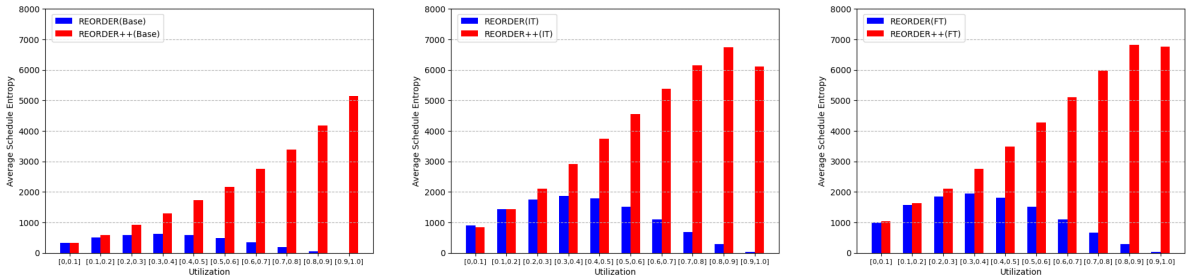
In order to evaluate the randomness of the schedule generated by different scheduling approaches with respect to the system utilization, we use the parameters similar to that in earlier research [15], [17], [35]. We randomly generated synthetic task sets evenly from ten base utilization groups [0.01+0.1*i, 0.09+0.1*i] for i = 0,...,9, where the base utilization of an instance is defined as the total utilization of the task set. For each base utilization group, there are four subgroups and each of them has a fixed number of tasks (i.e., 9, 11, 13 and 15), and the task utilizations are generated by the UUniFast [36] algorithm. All tasks are periodic real-time tasks with implicit deadlines,i.e., $D_i = T_i, \forall\tau_i$, and priorities are assigned according to the EDF scheduling algorithm. To generate uniform schedule entropy in each utilization group for ease of comparison, the period of a task is a divisor of 3, 000 and is randomly drawn from [1, 300], thus a common hyperperiod of 3, 000 is set over all the task sets. For each task τ_i, its execution time C_i is calculated as $C_i = \lceil U_i * T_i \rceil$ based on its utilization U_i and period T_i. For each subgroup, 100 task sets are generated, and thus 400 task sets are generated in each utilization group resulting in 4,000 task sets to test for each condition. The experiments use an implementation in Java and are performed on a desktop computer with an Intel(R) Core(TM) i7-9700 CPU @ 3.00 GHz and 16 GB of RAM.

C. Results

Schedule Entropy: Fig. 10 illustrates the average schedule entropies of REORDER++ and REORDER along with different randomization schemes, and Fig. 11 shows the schedule entropies of REORDER++, REORDER and vanilla EDF (i.e., no randomization) with different randomization schemes and different number of tasks, for each utilization group.

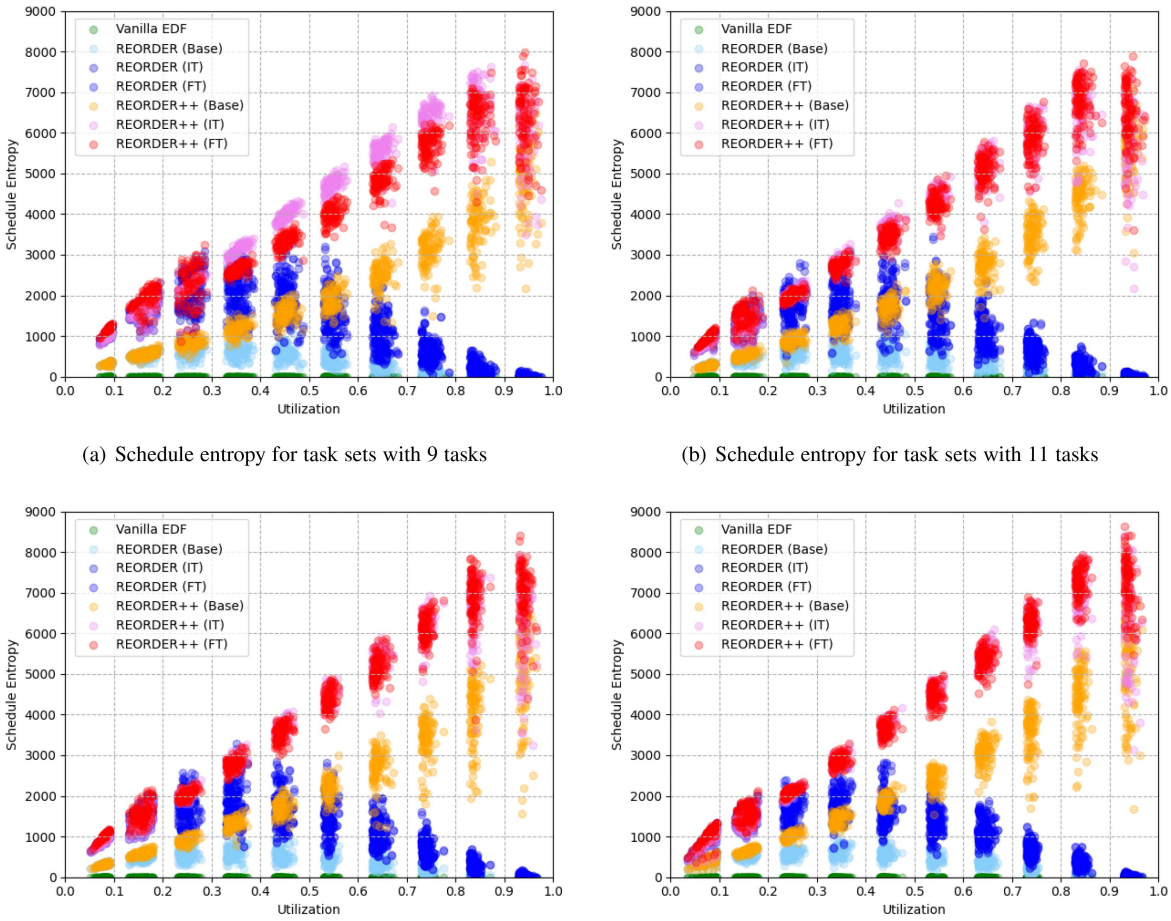
We begin with a few observations about the existing algorithms. First, the schedule entropy of vanilla EDF is very close or to zero. This is because the schedule under vanilla EDF is the same or slightly changed in multi-hyperperiods. Second, REORDER(Base), REORDER(IT) and REORDER(FT) perform significantly more schedule entropy than EDF in most cases. This is because REORDER can randomly execute a job by priority inversion, which increases the randomness of the schedule. Third, owing to the idle time scheduling, the schedule entropy of REORDER(IT) and REORDER(FT) are obviously larger than REORDER(Base), respectively. This is because the randomization of idle time makes the scheduling process more chaotic. Fourth, with the increase in the system load, the schedule entropy of REORDER(Base), REORDER(IT) and REORDER(FT) are increased first and then decreased. The possible reason is that, under the condition of low system utilization, large number of slack times in the system causes a low schedule entropy, as the system utilization increases, there are less opportunities for REORDER to introduce randomness into the schedule due to the pessimism of static analysis for the worst-case inversion budget.

Our main results is that REORDER++(Base), REORDER++(IT) and REORDER++(FT) consistently outperform REORDER(Base), REORDER(IT) and REORDER(FT). The main reason for this is that more randomness can be introduced in the schedule than REORDER owing to the online priority inversion test of REORDER++. REORDER++(IT) and



(a) Randomization with original task sets Γ (b) Randomization for augmented task sets $\Gamma \cup \{\tau_{idle}\}$ (c) Randomization for augmented task sets $\Gamma \cup \{\tau_{idle}\}$ with fine-grained switching

Fig. 10. The average schedule entropies gained by randomized scheduling strategies with different schemes.



(a) Schedule entropy for task sets with 9 tasks

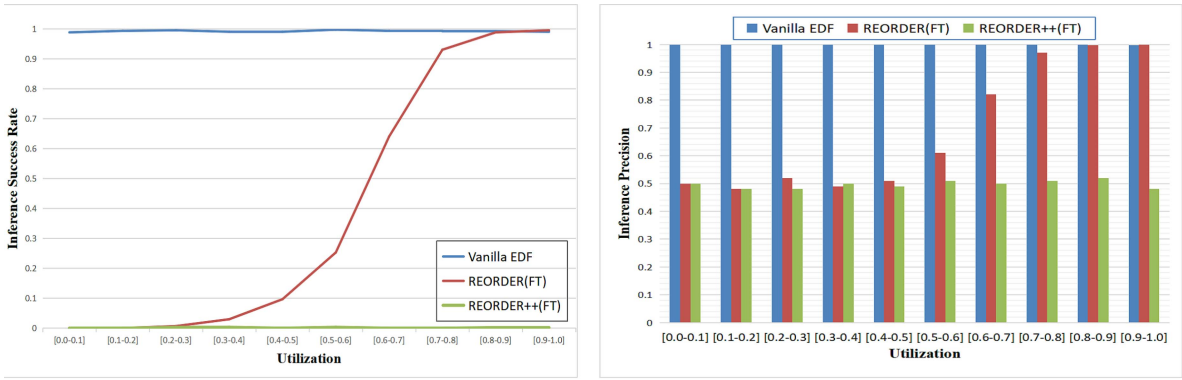
(b) Schedule entropy for task sets with 11 tasks

(c) Schedule entropy for task sets with 13 tasks

(d) Schedule entropy for task sets with 15 tasks

Fig. 11. The schedule entropy of task sets for varying number of tasks and utilizations.

REORDER++(FT) significantly perform more schedule entropy than REORDER++(Base). This is because the idle time scheduling and fine-grained switching effectively increase the randomness in the schedule. The performance gap tends to widen as the system load increases, depicting the wider availability of REORDER++ than REORDER. For the situation of low system load, such as task sets whose utilization is within [0.0,0.3], the degree of scheduling randomization of REORDER++ and REORDER with the same randomization schemes is close. This is because the performance of REORDER is little affected by a low system load with lots of slack times, hence, for both of the two algorithms, a large number of jobs can be used for randomization. For the situation of heavy system load, REORDER++ can still realize high-degree scheduling randomization, and this is because the performance of REORDER++ is not affected by the system load, while REORDER can only realize few



(a) Inference Success Rate (b) Inference Precision

Fig. 12. Results of the inference precision and success rate produced by the DyPs attack against vanilla EDF, REORDER and REORDER++ scheduling strategies.

randomization with less feasible jobs for randomization. In addition, under the situation of similar utilization, with the number of tasks of task sets increases, the performance gap between REORDER++ and REORDER also tends to widen. This is because there are more opportunities for REORDER++ to generate highly diverse feasible temporal execution schedule with more tasks.

DyPs Attack Defense Effect. Fig. 12 shows the inference precision and success rate of DyPs attack under different system utilization and scheduling strategies. First, we can see that DyPs attack always have a high inference precision and success rate of nearly 1 under EDF scheduling. This is because the scheduling process of EDF is always highly repetitive which effectively helps the success of the DyPs attack. Second, in the low system utilization, the inference success rate and precision of DyPs under REORDER(FT) is about 0 and 0.5 respectively, which implies that REORDER can effectively defend against the DyPs attack. This is because REORDER can realize effective randomized schedule under low system utilization. However, with the further increase of system utilization, the inference precision and success rate of REORDER are gradually improving, because REORDER can no longer achieve effective scheduling randomization. Note that, we can find that the inference success rate and precision of DyPs under REORDER++(FT) are close to 0 and 0.5 under all system utilization conditions, respectively, which implies REORDER++ can always effectively defend against DyPs attack. The reason is that REORDER++ effectively enhances the randomization degree of task scheduling by the online priority inversion test, so as to increases its defense performance.

Context Switching Cost. To evaluate the context switching cost of REORDER++, we measured the average number of context switches of vanilla EDF and REORDER++ with idle time scheduling and different job execution size (i.e., job execution time ω_i^t) in each hyperperiod. As shown in Fig. 13, with the job execution size increases, the average number of context switches of REORDER++ reduces. When the job is executed with maximum size (i.e., WCET), the average number of context switches of REORDER++ is almost the same as that of vanilla EDF. This is because the interruption frequency in scheduling

Fig. 13. The average number of context switches of REORDER++ with different job execution size and vanilla EDF.

is the main factor that affects the number of context switches in system. Therefore, the trade off between the context switching cost and the randomization performance can be achieved for REORDER++ by controlling the job execution size in randomized schedule.

VI. Conclusion

In order to ensure the functional correctness for the system, the real-time systems usually operate in a deterministic manner. However, this deterministic property often makes the real-time system vulnerable to timing side-channel attacks. In this paper, we proposed an efficient randomized scheduling strategy (named REORDER++) against side-channel attacks in dynamic-priority real-time systems that is based on the online priority inversion test. In our approach, we present an online absolute busy interval analysis method and determine the feasible priority inversion area at run-time for schedulability analysis. Owing to the online feasible priority inversion area analysis, the diversity of the feasible temporal execution schedule can be enhanced by considering the run-time information of the

system. Our evaluation shows that REORDER++ consistently outperforms the existing randomized scheduling method for dynamic-priority real-time systems.

REFERENCES

[1] Y. Xie, G. Zeng, R. Kurachi, F. Xiao, and H. Takada, "Optimizing extensibility of CAN FD for automotive cyber-physical systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 22, no. 12, pp. 7875–7886, Dec. 2021.

[2] Y. Xie, Y. Zhou, J. Xu, J. Zhou, X. Chen, and F. Xiao, "Cybersecurity protection on in-vehicle networks for distributed automotive cyber-physical systems: State-of-the-art and future challenges," *Softw.-Pract. Experience*, vol. 51, no. 11, pp. 2108–2127, Nov. 2021.

[3] D. Trilla, C. Hernandez, J. Abella, and F. J. Cazorla, "Cache side-channel attacks and time-predictability in high-performance critical real-time systems," in *Proc. 55th Annu. Des. Automat. Conf.*, 2018, pp. 1–6.

[4] Y. Lyu and P. Mishra, "A survey of side-channel attacks on caches and countermeasures," *J. Hardware Syst. Secur.*, vol. 2, no. 1, pp. 33–50, 2018.

[5] M. A. Aguida and M. Hasan, "Work in progress: Exploring schedule-based side-channels in trustzone-enabled real-time systems," in *Proc. IEEE 28th Real-Time Embedded Technol. Appl. Symp.*, 2022, pp. 301–304.

[6] C.-Y. Chen, S. Mohan, R. Pellizzoni, R. B. Bobba, and N. Kiyavash, "A novel side-channel in real-time schedulers," in *Proc. IEEE Real-Time Embedded Technol. Appl. Symp.*, 2019, pp. 90–102.

[7] C.-Y. Chen, S. Mohan, R. Pellizzoni, and R. B. Bobba, "On scheduler side-channels in dynamic-priority real-time systems," 2020, *arXiv:2001.06519*.

[8] S. Bi and Y. J. Zhang, "False-data injection attack to control real-time price in electricity market," in *Proc. IEEE Glob. Commun. Conf.*, 2013, pp. 772–777.

[9] M. A. Rahman and H. Mohsenian-Rad, "False data injection attacks with incomplete information against smart power grids," in *Proc. IEEE Glob. Commun. Conf.*, 2012, pp. 3153–3158.

[10] P. Milo, "They could hack-jack your car," *EE- Eval. Eng.*, vol. 50, no. 5, pp. 6–7, 2011.

[11] M. Luo, A. C. Myers, and G. E. Suh, "Stealthy tracking of autonomous vehicles with cache side channels," in *Proc. 29th USENIX Secur. Symp., USENIX Assoc.*, 2020, pp. 859–876. [Online]. Available: https://www.usenix.org/conference/usenixsecurity20/presentation/luo

[12] X. Liu, D. Liu, J. Zhang, T. Gu, and K. Li, "RFID and camera fusion for recognition of human-object interactions," in *Proc. 27th Annu. Int. Conf. Mobile Comput. Netw.*, 2021, pp. 296–308, doi: 10.1145/3447993.3483244.

[13] C. Lei, H.-Q. Zhang, J.-L. Tan, Y.-C. Zhang, and X.-H. Liu, "Moving target defense techniques: A survey," *Secur. Commun. Netw.*, vol. 2018, pp. 1–25, 2018.

[14] A. Bajic and G. T. Becker, "A critical view on moving target defense and its analogies," in *Proc. 17th ACM Int. Conf. Comput. Front.*, 2020, pp. 277–283.

[15] M.-K. Yoon, S. Mohan, C.-Y. Chen, and L. Sha, "TaskShuffler: A schedule randomization protocol for obfuscation against timing inference attacks in real-time systems," in *Proc. IEEE Real-Time Embedded Technol. Appl. Symp.*, 2016, pp. 1–12.

[16] M.-K. Yoon, J.-E. Kim, R. Bradford, and Z. Shao, "Timedice: Schedulability-preserving priority inversion for mitigating covert timing channels between real-time partitions," in *Proc. IEEE/IFIP 52nd Annu. Int. Conf. Dependable Syst. Netw.*, 2022, pp. 453–465.

[17] C.-Y. Chen, M. Hasan, A. Ghassami, S. Mohan, and N. Kiyavash, "Reorder: Securing dynamic-priority real-time systems using schedule obfuscation," 2018, *arXiv:1806.01393*.

[18] K. Krüger, M. Völp, and G. Fohler, "Vulnerability analysis and mitigation of directed timing inference based attacks on time-triggered systems," in *Proc. 30th Euromicro Conf. Real-Time Syst.*, (Leibniz International Proceedings in Informatics Series), S. Altmeyer, Ed., 2018, pp. 22:1–22:17. [Online]. Available: http://drops.dagstuhl.de/opus/volltexte/2018/8981

[19] C.-Y. Chen, "Scheduler side-channels in preemptive real-time systems: Attack and defense techniques," Ph.D. dissertation, Univ. Illinois at Urbana-Champaign, Champaign, IL, USA, 2020.

[20] K. H. Kim, "Application of deep learning for predicting schedules in real-time systems," Ph.D. dissertation, Univ. Illinois at Urbana-Champaign, Champaign, IL, USA, 2018.

[21] S. Liu, N. Guan, D. Ji, W. Liu, X. Liu, and W. Yi, "Leaking your engine speed by spectrum analysis of real-time scheduling sequences," *J. Syst. Archit.*, vol. 97, pp. 455–466, 2019.

[22] W. Jiang, Z. Song, J. Zhan, D. Liu, and J. Wan, "Layer-wise security protection for deep neural networks in industrial cyber physical systems," *IEEE Trans. Ind. Informat.*, vol. 18, no. 12, pp. 8797–8806, Dec. 2022.

[23] W. Jiang, Z. Song, J. Zhan, Z. He, X. Wen, and K. Jiang, "Optimized co-scheduling of mixed-precision neural network accelerator for real-time multitasking applications," *J. Syst. Archit.*, vol. 110, 2020, Art. no. 101775.

[24] M. Nasri, T. Chantem, G. Bloom, and R. M. Gerdes, "On the pitfalls and vulnerabilities of schedule randomization against schedule-based attacks," in *Proc. IEEE Real-Time Embedded Technol. Appl. Symp.*, 2019, pp. 103–116.

[25] M. Chougule and S. A. Soman, "Real-time data-assisted replay attack detection in wide-area protection system," *IET Gener., Transmiss. Distrib.*, vol. 14, no. 19, pp. 4021–4032, 2020.

[26] S. Liu and W. Yi, "Task parameters analysis in schedule-based timing side-channel attack," *IEEE Access*, vol. 8, pp. 157103–157115, 2020.

[27] T. Dayaratne, C. Rudolph, A. Liebman, M. Salehi, and S. He, "High impact false data injection attack against real-time pricing in smart grids," in *Proc. IEEE PES Innov. Smart Grid Technol. Europe*, 2019, pp. 1–5.

[28] C. Delimitrou and C. Kozyrakis, "Bolt: I know what you did last summer...in the cloud," *ACM SIGARCH Comput. Archit. News*, vol. 45, no. 1, pp. 599–613, 2017.

[29] R. Nathuji, A. Kansal, and A. Ghaffarkhah, "Q-clouds: Managing performance interference effects for QOS-aware clouds," in *Proc. 5th Eur. Conf. Comput. Syst.*, 2010, pp. 237–250.

[30] J. Chen et al., "Schedguard: Protecting against schedule leaks using linux containers," in *Proc. IEEE 27th Real-Time Embedded Technol. Appl. Symp.*, 2021, pp. 14–26.

[31] K. Krüger, N. Vreman, R. Pates, M. Maggio, M. Volp, and G. Fohler, "Randomization as mitigation of directed timing inference based attacks on time-triggered real-time systems with task replication," *LITES: Leibnitz Trans. Embedded Syst.*, vol. 7, pp. 01:1–01:29, 2021.

[32] C. L. Liu and J. W. Layland, "Scheduling algorithms for multiprogramming in a hard-real-time environment," *J. ACM*, vol. 20, no. 1, pp. 46–61, 1973.

[33] H. Jafarnejadsani, H. Lee, N. Hovakimyan, and P. Voulgaris, "Dual-rate L1 adaptive controller for cyber-physical sampled-data systems," in *Proc. IEEE 56th Annu. Conf. Decis. Control*, 2017, pp. 6259–6264.

[34] D. Zhai et al., "Parametric local multiview hamming distance metric learning," *Pattern Recognit.*, vol. 75, pp. 250–262, 2018.

[35] S. Mohan, M. K. Yoon, R. Pellizzoni, and R. Bobba, "Real-time systems security through scheduler constraints," in *Proc. IEEE 26th Euromicro Conf. Real-Time Syst.*, 2014, pp. 129–140.

[36] E. Bini and G. C. Buttazzo, "Measuring the performance of schedulability tests," *Real-Time Syst.*, vol. 30, no. 1/2, pp. 129–154, 2005.

Jiankang Ren (Member, IEEE) received the B.Sc., M.E., and Ph.D. degrees in computer science from the Dalian University of Technology, Dalian, China, in 2008, 2011, and 2015, respectively. From September 2013 to September 2014, he was a Visiting Scholar with the Computer and Information Science Department, University of Pennsylvania, Philadelphia, PA, USA. He is currently an Associate Professor with the School of Computer Science and Technology, Dalian University of Technology. He has authored more than 50 scientific papers in several journals and conferences including DAC, INFOCOM, DATE, ICNP, ICDCS, ICWS, ECRTS, ACM Multimedia, IEEE TITS, JSS, and JSA. His research interests include cyber-physical systems, intelligent autonomous systems, and swarm intelligent systems. In 2020, he was the recipient of the ACM Rising Star Award.

Zheng Wang received the B.S. degree in computer science and technology from Dalian Maritime University, Dalian, China, in 2020. He is currently working toward the M.S. degree with the School of Computer Science and Technology, Dalian University of Technology, Dalian. His research focuses on embedded real-time system security.

Chi Lin (Senior Member, IEEE) received the B.E. and Ph.D. degrees from the Dalian University of Technology, Dalian, China, in 2008 and 2013, respectively. Since 2014, he has been an Assistant Professor with the School of Software, Dalian University of Technology. Since 2017, he has been an Associate Professor with the School of Software, Dalian University of Technology. He has authored more than 70 scientific papers in several journals and conferences including Mobicom, INFOCOM, ICNP, ICDCS, SECON, ICPP, IEEE/ACM TON, TMC, ACM TECS, IEEE TVT, and special issue of *SCIENCE*. His research interests include pervasive computing, cyber-physical systems, and wireless sensor networks. In 2015, he was the recipient of the ACM Academic Rising Star. He is a Senior Member of the China Computer Federation.

Mohammad S. Obaidat (Life Fellow, IEEE) received the M.S. and Ph.D. degrees in computer engineering from Ohio State University, Columbus, OH, USA. He is currently an internationally well-known Academic/Researcher/Scientist. Among his previous positions, as an Advisor to President of Philadelphia University, Jordan, the President of SCS, Dean of the College of Engineering, Prince Sultan University, Riyadh, Saudi Arabia, Chair and a Professor with the Department of CIS, Fordham University, New York, NY, USA, and the Chair and a Professor with Monmouth University, West Long Branch, NJ, USA. He is also the Founding Dean of the College of Computing and Informatics, University of Sharjah, Sharjah, UAE. He is also the PR of China Ministry of Education Distinguished Overseas Professor with the University of Science and Technology Beijing, Beijing, China, and an Honorary Distinguished Professor with the Amity University- A Global University. He has received extensive research funding and published to date more than 70 books, 70 Book Chapters, and about (1000) refereed articles in scholarly international journals and proceedings of international conferences. He is the Editor-in-Chief of three scholarly journals and Editor of many other international journals. He is the Founding Editor-in Chief of Wiley *Security and Privacy* Journal. Moreover, he is the Founder or Co-founder of five International Conferences. He has chaired numerous (more than 160) international conferences and has given numerous (more than 160) keynote speeches worldwide. He founded or co-founded five international conferences. He was an ABET/CSAB Evaluator and served on IEEE CS Fellow Evaluation Committee. He was the recipient of many best paper awards for his papers, including ones from IEEE ICC, IEEE Globecom, AICSA, CITS, SPECTS, and DCNET International conferences, Best Paper awards from IEEE Systems Journal in 2018 and in 2019 (two Best Paper Awards), and four best paper awards from IEEE SYSTEMS JOURNAL. He also received many other worldwide awards for his technical contributions, including: The 2018 IEEE ComSoc-Technical Committee on Communications Software 2018 Technical Achievement Award for contribution to Cybersecurity, Wireless Networks Computer Networks and Modeling and Simulation, SCS prestigious McLeod Founder's Award, Presidential Service Award, SCS Hall of Fame Lifetime Achievement Award for his technical contribution to modeling and simulation and for his outstanding visionary leadership and dedication to increasing the effectiveness and broadening the applications of modeling and simulation worldwide, SCS Outstanding Service Award, and IEEE CITS Hall of Fame Distinguished and Eminent Award. He is a Fellow of SCS.

Hongrui Xie is currently working toward the B.S. degree with the School of Computer Science and Technology, Dalian University of Technology, Dalian, China. His research focuses on embedded real-time system security.

Haihui Zhu is currently working toward the B.S. degree with the School of Electronic Information and Electrical Engineering, University of Science and Technology Dalian, Dalian, China. His research focuses on embedded real-time system security.

Chunxiao Liu received the B.S. degree from the School of Chemical Engineering, Dalian University of Technology, Dalian, China, in 2021. He is currently working toward the M.S. degree with the School of Computer Science and Technology, Dalian University of Technology, Dalian, China. His research focuses on embedded real-time system security.

Kaiwen Wang is currently working toward the B.S. degree with the school of Computer Science and Technology, Dalian University of Technology, Dalian, China. His research focuses on embedded real-time system security.

Guozhen Tan received the B.S. degree from the Shenyang University of Technology, Shenyang, China, the M.S. degree from the Harbin Institute of Technology, Harbin, China, and the Ph.D. degree from the Dalian University of Technology, Dalian, China. From 2007 to 2008, he was a Visiting Scholar with the Department of Electrical and Computer Engineering, University of Illinois at Urbana–Champaign, Urbana, IL, USA. He is currently a Professor with the School of Computer Science and Technology, Dalian University of Technology. His research interests include the Internet of Things, cyber-physical systems, vehicular ad hoc networks, intelligent transportation systems, and network optimization algorithms.